

Tidal Deformability Code

Physics and Numerical Documentation

Documentation of the neutron-star structure and tidal-deformability code

Program:

`logtov_seq_geom_tidal.f90`

Author:

Ayush Fotedar

Date:

October 1, 2026

Contents

1	Purpose of the Program	4
2	The Equation of State	5
3	Static Spherically Symmetric Neutron Stars	6
4	The Tolman–Oppenheimer–Volkoff Equations	6
4.1	Pressure Equation	6
4.2	Mass Equation	7
5	Metric Potential	8
6	Rest Mass of the Star	8
7	Constructing a Stellar Model	9
8	Numerical Integration	10
9	Tidal Deformation	11
10	Tidal Metric Perturbation	12
11	Central Boundary Conditions for the Tidal Perturbation	13
12	Equation-of-State Dependence of the Tidal Equation	13
13	Surface Tidal Quantity	14
14	Compactness	14
15	Quadrupolar Tidal Love Number	15
16	Dimensionless Tidal Deformability	15
17	Relationship Between the Physics and the Fortran Code	16
18	Overall Computational Workflow	16
19	Input File	18
20	Compiling the Program	18

21 Running the Program	19
22 Output Quantities	19
23 Mass–Radius and Tidal Sequences	20
24 Application to Mass–Radius Curvature	20
25 Numerical Considerations	21
25.1 Integration Step Size	21
25.2 Central-Density Resolution	21
25.3 Low-Mass Models	21
26 Summary	21
27 Code Architecture and Numerical Implementation	22
27.1 Overview	22
27.2 Repository Structure	22
27.3 Program Input	23
27.4 Main Program Variables	24
27.5 Equation-of-State Arrays	25
27.6 Equation-of-State Initialisation	25
27.7 EoS Interpolation	26
27.7.1 Energy Density as a Function of Pressure	26
27.7.2 Pressure as a Function of Energy Density	27
27.8 Central-Density Sequence	27
27.9 Initial Conditions	28
27.10 Initial Tidal Perturbation	29
27.11 Differential-Equation Routines	29
27.12 Tidal EoS Derivative	30
27.13 Fourth-Order Runge–Kutta Integration	30
27.14 Surface and Termination Condition	31
27.15 Metric Normalisation	32
27.16 Calculation of the Tidal Surface Variable	32
27.17 Compactness	32
27.18 Love Number	33
27.19 Dimensionless Tidal Deformability	33

27.20	Output File	33
27.21	Auxiliary Output Files	35
27.22	Program Flow	35
27.23	Compilation	36
27.24	Running the Code	36
27.25	Python Analysis	37
27.26	Important Implementation Notes	37
27.26.1	EoS Header Handling	37
27.26.2	Low-Mass Tidal Models	38
27.26.3	Output Overwriting	38
27.26.4	Radial Step Size	38
27.27	Suggested Future Improvements	38
27.28	Summary of Code Execution	39
28	References	39

1 Purpose of the Program

The purpose of the program `logtov_seq_geom_tidal.f90` is to construct sequences of static, spherically symmetric neutron-star models for a supplied equation of state (EoS), and to calculate their bulk and tidal properties.

For a chosen EoS, the program considers a sequence of different central energy densities,

$$\epsilon_c^{(1)}, \epsilon_c^{(2)}, \dots, \epsilon_c^{(N)}, \quad (1.1)$$

and integrates the stellar-structure equations independently for each value.

Each central energy density therefore corresponds to a different equilibrium neutron-star configuration.

For every successful stellar model, the program determines quantities including

$$M, \quad R, \quad M_{\text{rest}}, \quad C, \quad k_2, \quad \kappa_2, \quad \Lambda, \quad (1.2)$$

where

- M is the gravitational mass,
- R is the stellar radius,
- M_{rest} is the rest or baryonic mass,
- $C = M/R$ is the compactness in geometrized units,
- k_2 is the quadrupolar tidal Love number,
- κ_2 is a tidal coupling parameter,
- Λ is the dimensionless tidal deformability.

Consequently, a run over many central densities produces an entire stellar sequence from which relations such as

$$M(R), \quad (1.3)$$

$$\Lambda(M), \quad (1.4)$$

$$k_2(M), \quad (1.5)$$

and

$$C(M) \quad (1.6)$$

can be constructed.

Conceptually, the program follows the sequence

$$\boxed{\text{Equation of State} \longrightarrow \text{TOV stellar structure} \longrightarrow \text{tidal perturbation} \longrightarrow \{M, R, k_2, \Lambda, \dots\}} \quad (1.7)$$

for every chosen central density.

2 The Equation of State

A neutron star is supported against gravitational collapse by the pressure of extremely dense matter.

The microscopic properties of this matter are encoded in an equation of state.

For the stellar-structure problem, the fundamental relation is

$$P = P(\epsilon), \quad (2.1)$$

where P is the pressure and ϵ is the total energy density.

During the numerical integration, it must also be possible to evaluate

$$\epsilon = \epsilon(P). \quad (2.2)$$

The EoS tables supplied with the current code contain quantities of the form

$$\epsilon, \quad P, \quad n_B, \quad \mu_B, \quad (2.3)$$

where

$$\epsilon : \text{total energy density}, \quad (2.4)$$

$$P : \text{pressure}, \quad (2.5)$$

$$n_B : \text{baryon number density}, \quad (2.6)$$

$$\mu_B : \text{baryon chemical potential}. \quad (2.7)$$

The nominal units of the tabulated quantities are

$$\epsilon : \text{MeV fm}^{-3}, \quad (2.8)$$

$$P : \text{MeV fm}^{-3}, \quad (2.9)$$

$$n_B : \text{fm}^{-3}, \quad (2.10)$$

$$\mu_B : \text{MeV}. \quad (2.11)$$

The relationship between pressure and energy density is required for the TOV integration, while the baryon-density information is used in the calculation of rest-mass quantities.

3 Static Spherically Symmetric Neutron Stars

The program assumes that each stellar configuration is

- static,
- non-rotating,
- spherically symmetric,
- isolated,
- described by general relativity.

Under these assumptions, the spacetime metric may be written in Schwarzschild-like coordinates as

$$ds^2 = -e^{2\nu(r)} dt^2 + \left(1 - \frac{2m(r)}{r}\right)^{-1} dr^2 + r^2 d\Omega^2, \quad (3.1)$$

when geometrized units,

$$G = c = 1, \quad (3.2)$$

are used.

Here,

$$m(r) \quad (3.3)$$

is the gravitational mass enclosed inside radius r , while

$$\nu(r) \quad (3.4)$$

is a gravitational metric potential.

The stellar matter is approximated as a perfect fluid with stress-energy tensor

$$T^{\mu\nu} = (\epsilon + P)u^\mu u^\nu + Pg^{\mu\nu}. \quad (3.5)$$

Einstein's field equations together with hydrostatic equilibrium then lead to the Tolman–Oppenheimer–Volkoff equations.

4 The Tolman–Oppenheimer–Volkoff Equations

4.1 Pressure Equation

The relativistic equation for hydrostatic equilibrium is

$$\boxed{\frac{dP}{dr} = -\frac{(\epsilon + P)(m + 4\pi r^3 P)}{r(r - 2m)}} \quad (4.1)$$

in geometrized units.

An equivalent expression with G and c shown explicitly is

$$\frac{dP}{dr} = -\frac{G\epsilon m}{r^2} \left(1 + \frac{P}{\epsilon c^2}\right) \left(1 + \frac{4\pi r^3 P}{mc^2}\right) \left(1 - \frac{2Gm}{rc^2}\right)^{-1}. \quad (4.2)$$

This differential equation is implemented in the Fortran function

`rhsp(rr,pp,mm).`

Thus,

$$\boxed{\text{rhsp} \longleftrightarrow \frac{dP}{dr}} \quad (4.3)$$

The three relativistic correction factors have clear physical interpretations.

The factor

$$1 + \frac{P}{\epsilon c^2} \quad (4.4)$$

accounts for the contribution of pressure to inertial mass.

The factor

$$1 + \frac{4\pi r^3 P}{mc^2} \quad (4.5)$$

accounts for the contribution of pressure to the active gravitational field.

Finally,

$$\left(1 - \frac{2Gm}{rc^2}\right)^{-1} \quad (4.6)$$

is a relativistic spacetime-curvature correction.

4.2 Mass Equation

The enclosed gravitational mass satisfies

$$\boxed{\frac{dm}{dr} = 4\pi r^2 \epsilon} \quad (4.7)$$

in geometrized units.

This equation is implemented in the function

$$\text{rhsm}(\text{rr}, \text{pp}, \text{mm}).$$

Therefore,

$$\boxed{\text{rhsm} \longleftrightarrow \frac{dm}{dr}} \quad (4.8)$$

At the stellar surface,

$$r = R, \quad (4.9)$$

the gravitational mass of the star is

$$\boxed{M = m(R)}. \quad (4.10)$$

5 Metric Potential

The code simultaneously evolves the metric potential $\nu(r)$.

Its differential equation is

$$\boxed{\frac{d\nu}{dr} = \frac{m + 4\pi r^3 P}{r(r - 2m)}} \quad (5.1)$$

in geometrized units.

This equation is implemented by

$$\text{rhsn}(\text{rr}, \text{pp}, \text{mm}).$$

Hence,

$$\boxed{\text{rhsn} \longleftrightarrow \frac{d\nu}{dr}} \quad (5.2)$$

After the integration reaches the stellar surface, the metric function must be normalized so that the interior solution joins consistently to the exterior Schwarzschild solution.

The corresponding normalization takes the form

$$\nu_0 = \ln \sqrt{1 - \frac{2M}{R}} - \nu(R). \quad (5.3)$$

6 Rest Mass of the Star

The gravitational mass M is not identical to the sum of the rest masses of the baryons that constitute the star.

The program therefore separately integrates the rest or baryonic mass,

$$M_{\text{rest}}. \quad (6.1)$$

The corresponding radial equation can be written as

$$\boxed{\frac{dM_{\text{rest}}}{dr} = 4\pi r^2 \rho_{\text{rest}} \left(1 - \frac{2m}{r}\right)^{-1/2}} \quad (6.2)$$

in geometrized units.

The factor

$$\left(1 - \frac{2m}{r}\right)^{-1/2} \quad (6.3)$$

accounts for the proper spatial volume in curved spacetime.

The equation is implemented by the function

$$\text{rhsr}(\text{rr}, \text{pp}, \text{mm}).$$

Therefore,

$$\boxed{\text{rhsr} \longleftrightarrow \frac{dM_{\text{rest}}}{dr}} \quad (6.4)$$

The difference between baryonic and gravitational mass,

$$M_{\text{rest}} - M, \quad (6.5)$$

is related to the gravitational binding energy of the neutron star.

7 Constructing a Stellar Model

A neutron-star solution is specified by selecting a central energy density

$$\epsilon_c. \quad (7.1)$$

The equation of state is then used to calculate the corresponding central pressure,

$$P_c = P(\epsilon_c). \quad (7.2)$$

In the program this operation is performed through the EoS interpolation routine

$$\text{pout}(\text{rho0}).$$

Conceptually,

$$\boxed{\epsilon_c \xrightarrow{\text{EoS}} P_c} \quad (7.3)$$

The numerical integration begins at a small but non-zero radius r_0 , rather than exactly at

$$r = 0, \quad (7.4)$$

because several terms in the differential equations contain factors such as $1/r$ or $1/r^2$.

Close to the centre, the enclosed mass is approximated by

$$m(r_0) \simeq \frac{4\pi}{3} \epsilon_c r_0^3. \quad (7.5)$$

The program then integrates outward until the pressure becomes sufficiently small.

The location where

$$P(R) \simeq 0 \quad (7.6)$$

defines the stellar surface.

The final radius and mass are then

$$R = r_{\text{surface}}, \quad (7.7)$$

$$M = m(R). \quad (7.8)$$

Repeating this procedure for many central densities generates a mass–radius sequence,

$$M(R). \quad (7.9)$$

8 Numerical Integration

The coupled differential equations are integrated using the classical fourth-order Runge–Kutta method.

For a generic differential equation

$$\frac{dy}{dr} = f(r, y), \quad (8.1)$$

the RK4 method evaluates

$$k_1 = f(r_n, y_n), \quad (8.2)$$

$$k_2 = f\left(r_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right), \quad (8.3)$$

$$k_3 = f\left(r_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right), \quad (8.4)$$

$$k_4 = f(r_n + h, y_n + hk_3). \quad (8.5)$$

The solution is then advanced using

$$\boxed{y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)} \quad (8.6)$$

The program applies this procedure simultaneously to

$$P, \quad m, \quad \nu, \quad M_{\text{rest}}, \quad H, \quad \beta. \quad (8.7)$$

The first four quantities describe the equilibrium stellar background, whereas H and β describe the tidal perturbation.

9 Tidal Deformation

A neutron star in a binary system experiences the gravitational field of its companion.

The companion generates an external quadrupolar tidal field,

$$\mathcal{E}_{ij}, \quad (9.1)$$

which induces a quadrupole moment in the neutron star,

$$Q_{ij}. \quad (9.2)$$

To linear order, the relationship is written as

$$\boxed{Q_{ij} = -\lambda \mathcal{E}_{ij}} \quad (9.3)$$

where λ is the dimensional tidal deformability.

A larger value of λ corresponds to a star that is more easily deformed by an external gravitational field.

A smaller value corresponds to a more compact and less deformable object.

Because the neutron-star radius and internal density profile depend on the equation of state, tidal deformability provides a direct probe of dense nuclear matter.

10 Tidal Metric Perturbation

The equilibrium TOV solution describes a perfectly spherical star.

To determine its response to an external quadrupolar tidal field, a small $l = 2$ perturbation is added to the background spacetime.

The radial part of this perturbation may be described using a function

$$H(r). \tag{10.1}$$

The second-order differential equation for H can be rewritten as two coupled first-order equations by defining

$$\boxed{\beta(r) = \frac{dH}{dr}} \tag{10.2}$$

so that

$$\frac{dH}{dr} = \beta. \tag{10.3}$$

This equation is represented in the code by

$$\text{rhsh}(\dots).$$

The evolution equation for β ,

$$\frac{d\beta}{dr}, \tag{10.4}$$

is implemented by

$$\text{rhsb}(\dots).$$

Therefore,

$$\text{rhsh} \longleftrightarrow \frac{dH}{dr}, \tag{10.5}$$

$$\text{rhsb} \longleftrightarrow \frac{d\beta}{dr}. \tag{10.6}$$

Since

$$\beta = \frac{dH}{dr}, \tag{10.7}$$

the latter corresponds to the second derivative of the perturbation,

$$\frac{d\beta}{dr} = \frac{d^2H}{dr^2}. \tag{10.8}$$

11 Central Boundary Conditions for the Tidal Perturbation

Regularity at the centre requires the quadrupolar perturbation to behave as

$$H(r) \propto r^2 \quad (11.1)$$

for the $l = 2$ mode.

The initial conditions are therefore chosen in the form

$$H(r_0) = a_0 r_0^2 \quad (11.2)$$

and

$$\beta(r_0) = 2a_0 r_0. \quad (11.3)$$

The constant a_0 fixes only the arbitrary normalization of the perturbation.

The tidal Love number depends on ratios involving H and H' , and is therefore independent of the overall amplitude.

The code can consequently choose

$$a_0 = 1. \quad (11.4)$$

12 Equation-of-State Dependence of the Tidal Equation

The tidal perturbation equation requires the derivative

$$\frac{d\epsilon}{dP}. \quad (12.1)$$

In the current implementation, this derivative is estimated numerically using nearby pressure values,

$$\boxed{\frac{d\epsilon}{dP} \simeq \frac{\epsilon(1.01P) - \epsilon(0.99P)}{0.02P}} \quad (12.2)$$

This quantity is closely related to the sound speed.

In relativistic units,

$$c_s^2 = \frac{dP}{d\epsilon}, \quad (12.3)$$

and therefore

$$\frac{d\epsilon}{dP} = \frac{1}{c_s^2}. \quad (12.4)$$

The microscopic EoS therefore enters the tidal calculation not only through the equilibrium stellar structure but also directly through the local compressibility of the matter.

13 Surface Tidal Quantity

Once the stellar surface has been reached, the program constructs a dimensionless surface quantity conventionally denoted by y .

The code evaluates a quantity of the form

$$y = \frac{R\beta(R)}{H(R)} - \frac{4\pi R^3 \epsilon(R)}{M} \quad (13.1)$$

where

$$\beta(R) = H'(R). \quad (13.2)$$

The first term is therefore the logarithmic derivative of the tidal perturbation,

$$\frac{RH'(R)}{H(R)}. \quad (13.3)$$

The variables

$$M, \quad R, \quad y \quad (13.4)$$

are then used to calculate the quadrupolar tidal Love number.

14 Compactness

The relativistic compactness of a neutron star is defined as

$$C = \frac{GM}{Rc^2} \quad (14.1)$$

In geometrized units,

$$G = c = 1, \quad (14.2)$$

and therefore

$$C = \frac{M}{R}. \quad (14.3)$$

Compactness is particularly important for tidal deformability because the latter scales approximately with the fifth inverse power of C .

15 Quadrupolar Tidal Love Number

The quadrupolar tidal Love number k_2 measures the quadrupolar response of the neutron star to an external tidal field.

Once the compactness C and surface quantity y have been calculated, the Love number can be written as

$$k_2 = \frac{\frac{8}{5}C^5(1-2C)^2[2+2C(y-1)-y]}{D} \quad (15.1)$$

where

$$\begin{aligned} D = & 2C[6-3y+3C(5y-8)] \\ & + 4C^3[13-11y+C(3y-2)+2C^2(1+y)] \\ & + 3(1-2C)^2[2-y+2C(y-1)]\ln(1-2C). \end{aligned} \quad (15.2)$$

The value of k_2 depends on both the compactness and the internal structure of the star.

16 Dimensionless Tidal Deformability

The dimensional tidal deformability λ is conventionally related to the Love number through

$$\lambda = \frac{2}{3} \frac{k_2 R^5}{G}. \quad (16.1)$$

For gravitational-wave calculations, the dimensionless tidal deformability is generally more convenient.

It is defined as

$$\Lambda = \frac{\lambda c^{10}}{G^4 M^5}. \quad (16.2)$$

Combining this definition with the compactness gives

$$\Lambda = \frac{2}{3} k_2 C^{-5} \quad (16.3)$$

Since

$$C = \frac{GM}{Rc^2}, \quad (16.4)$$

one may write schematically

$$\Lambda \propto k_2 \left(\frac{Rc^2}{GM} \right)^5. \quad (16.5)$$

This extremely strong radius dependence makes tidal deformability particularly sensitive to the equation of state.

At a fixed mass, a neutron star with a larger radius will generally have a much larger value of Λ .

Conversely, massive and compact neutron stars tend to have smaller tidal deformabilities.

17 Relationship Between the Physics and the Fortran Code

The most important quantities in the program can be mapped directly to their physical interpretation.

Fortran quantity	Physical meaning
rho0	Central energy density ϵ_c
p	Pressure $P(r)$
r	Radial coordinate r
m	Enclosed gravitational mass $m(r)$
mrest	Enclosed rest/baryonic mass
nue	Metric potential $\nu(r)$
caph	Tidal perturbation $H(r)$
beta	Tidal derivative dH/dr
rhsp	dP/dr
rhsm	dm/dr
rhsn	$d\nu/dr$
rhsr	dM_{rest}/dr
rhsh	dH/dr
rhsb	$d\beta/dr$
eps(p)	EoS interpolation $\epsilon(P)$
pout(e)	EoS interpolation $P(\epsilon)$
rest(p)	Rest-mass density interpolation
tidalk2	Quadrupolar Love number k_2
lambda	Dimensionless tidal deformability Λ
comp	Compactness M/R

Table 1: Relationship between important Fortran variables and physical quantities.

18 Overall Computational Workflow

A single stellar model follows the sequence

$$\epsilon_c \tag{18.1}$$

↓

$$P_c = P(\epsilon_c) \tag{18.2}$$

↓

The following equations are integrated simultaneously:

$$\left\{ \begin{array}{l} \frac{dP}{dr}, \\ \frac{dm}{dr}, \\ \frac{d\nu}{dr}, \\ \frac{dM_{\text{rest}}}{dr}, \\ \frac{dH}{dr}, \\ \frac{d\beta}{dr}. \end{array} \right. \tag{18.3}$$

The integration proceeds outward until approximately

$$P(R) = 0. \tag{18.4}$$

At the surface the code determines

$$M, \quad R, \quad C, \quad y. \tag{18.5}$$

These quantities are then used to calculate

$$k_2, \tag{18.6}$$

followed by

$$\boxed{\Lambda = \frac{2}{3}k_2C^{-5}.} \tag{18.7}$$

Thus, conceptually,

$$\boxed{\epsilon_c \rightarrow P_c \rightarrow \text{TOV} + \text{tidal integration} \rightarrow (M, R, C, y) \rightarrow k_2 \rightarrow \Lambda} \tag{18.8}$$

The entire procedure is repeated for every central density specified in the input file.

19 Input File

The program reads its basic configuration from the file

`infile.`

A typical input file is

```
1.e14    0.3e14    100
EoS_dd2_npY_T0_beta_eq_01_04_Mg_800.dat
```

The quantities on the first line correspond to

$$\rho_{c,\text{start}}, \quad \Delta\rho_c, \quad N, \quad (19.1)$$

where

- $\rho_{c,\text{start}}$ is the central density of the first model,
- $\Delta\rho_c$ is the central-density increment,
- N is the number of stellar configurations to calculate.

Therefore, the central density for model i is approximately

$$\rho_c^{(i)} = \rho_{c,\text{start}} + (i - 1)\Delta\rho_c. \quad (19.2)$$

The second line contains the filename of the EoS table.

20 Compiling the Program

The source can be compiled using the GNU Fortran compiler.

A standard compilation command is

```
gfortran -o logtov_seq_geom_tidal.out logtov_seq_geom_tidal.f90
```

If required, the code may instead be compiled using

```
gfortran -o logtov_seq_geom_tidal.out \  
logtov_seq_geom_tidal.f90 -fno-automatic
```

The resulting executable is

`logtov_seq_geom_tidal.out.`

21 Running the Program

Once the EoS file and `infile` are present in the working directory, the program is executed using

```
./logtov_seq_geom_tidal.out
```

The main output file is

```
logtov_seq_geom_tidal.dat.
```

This file contains the calculated properties for the sequence of neutron-star models.

22 Output Quantities

The current output contains quantities of the form

```
rho0, r, m, mrest, exp(nue), nue0, (m/r),  
restini, pini, tidalk2, kappa2, Lambda,  
y, eps(p)*6.176e17, H, beta, ef, nf
```

The most important columns for mass–radius and tidal-deformability analysis are summarized below.

Column	Quantity	Meaning
1	ρ_0	Central energy density
2	R	Stellar radius
3	M	Gravitational mass
4	M_{rest}	Rest/baryonic mass
7	M/R	Compactness
8	$\rho_{\text{rest},c}$	Central rest-mass density
9	P_c	Central pressure
10	k_2	Quadrupolar Love number
11	κ_2	Tidal coupling constant
12	Λ	Dimensionless tidal deformability
13	y	Surface tidal variable
15	H	Tidal perturbation
16	β	dH/dr

Table 2: Important quantities written to the main output file.

23 Mass–Radius and Tidal Sequences

After the program has been run over a sequence of central densities, the output can be used to construct several physically useful relations.

The standard mass–radius curve is

$$M(R). \quad (23.1)$$

The tidal deformability can be studied as a function of mass,

$$\Lambda(M), \quad (23.2)$$

and similarly,

$$k_2(M) \quad (23.3)$$

and

$$C(M) \quad (23.4)$$

can be investigated.

The maximum of the gravitational-mass sequence,

$$\frac{dM}{d\rho_c} = 0, \quad (23.5)$$

is of particular importance because it approximately marks the turning point between the stable and unstable branches of the non-rotating stellar sequence.

24 Application to Mass–Radius Curvature

For the present project, the generated mass–radius sequence can also be used to calculate the geometric curvature of the M – R relation.

Treating both mass and radius as functions of a parameter t , the curvature of the parametric curve

$$\mathbf{x}(t) = (R(t), M(t)) \quad (24.1)$$

is

$$\kappa(t) = \frac{|R'(t)M''(t) - M'(t)R''(t)|}{[R'(t)^2 + M'(t)^2]^{3/2}} \quad (24.2)$$

A natural parameter is the central density,

$$t = \rho_c. \quad (24.3)$$

The curvature may then be written as

$$\kappa(\rho_c) = \frac{\left| \frac{dR}{d\rho_c} \frac{d^2 M}{d\rho_c^2} - \frac{dM}{d\rho_c} \frac{d^2 R}{d\rho_c^2} \right|}{\left[\left(\frac{dR}{d\rho_c} \right)^2 + \left(\frac{dM}{d\rho_c} \right)^2 \right]^{3/2}} \quad (24.4)$$

Since second derivatives are involved, the central-density sequence should be sampled sufficiently finely to reduce numerical noise.

25 Numerical Considerations

Several numerical issues must be considered when interpreting the results.

25.1 Integration Step Size

The stellar equations are solved using a finite radial step size h .

A smaller value of h generally increases numerical accuracy but also increases computation time.

A larger value speeds up the integration but may reduce the accuracy of quantities such as

$$R, \quad M, \quad k_2, \quad \Lambda. \quad (25.1)$$

This is particularly important when numerical derivatives of the mass–radius sequence are required.

25.2 Central-Density Resolution

If the central-density increment $\Delta\rho_c$ is too large, sharp features in the mass–radius relation may not be sufficiently resolved.

This can strongly affect numerical first and second derivatives.

For curvature calculations, a finer central-density grid is therefore preferable.

25.3 Low-Mass Models

The supplied code documentation notes that tidal quantities may behave unusually for very low-mass stellar models.

Such models should therefore be treated carefully when constructing tidal-deformability curves.

26 Summary

The code combines three main physical ingredients:

1. A dense-matter equation of state,

2. The Tolman–Oppenheimer–Volkoff equations,
3. The relativistic quadrupolar tidal-perturbation equations.

For every central density, the code first determines the central pressure from the EoS.

It then integrates

$$P(r), \quad m(r), \quad \nu(r), \quad M_{\text{rest}}(r), \quad H(r), \quad \beta(r) \quad (26.1)$$

from the stellar centre to the surface.

The resulting mass, radius and tidal perturbation determine

$$C, \quad k_2, \quad \Lambda. \quad (26.2)$$

The complete computational sequence may therefore be summarized as

$$\boxed{\text{EoS} \rightarrow \epsilon_c \rightarrow P_c \rightarrow \text{TOV integration} \rightarrow M, R \rightarrow \text{tidal integration} \rightarrow k_2 \rightarrow \Lambda} \quad (26.3)$$

Repeating the calculation over many central densities produces the full neutron-star sequence required for mass–radius, tidal-deformability, and geometric-curvature studies.

27 Code Architecture and Numerical Implementation

27.1 Overview

The neutron-star structure and tidal-deformability calculation is implemented in the Fortran program

`logtov_seq_geom_tidal.f90.`

The program constructs a sequence of neutron-star models for a specified equation of state (EoS). Each model corresponds to a different central energy density. For every central density, the program simultaneously integrates the stellar-structure and tidal-perturbation equations from the centre of the star to its surface.

The principal computational sequence is

$$\boxed{\text{infile} \rightarrow \text{EoS initialization} \rightarrow \rho_c \rightarrow P_c \rightarrow \text{RK4 integration} \rightarrow M, R \rightarrow k_2, \Lambda \rightarrow \text{output}} \quad (27.1)$$

The calculation is repeated for the number of stellar models specified by the user.

27.2 Repository Structure

The code is organised in the repository approximately as

```

tov-tidal-deformability/
|
|-- README.md
|-- infile
|
|-- src/
|   '-- logtov_seq_geom_tidal.f90
|
|-- eos/
|   |-- lists/
|   '-- EoS_dd2_npY_T0_beta_eq_eta_D-eta_V-Mg/
|
|-- analysis/
|   '-- analyse_tidal.py
|
|-- scripts/
|   '-- Tidal_control.sh
|
|-- docs/
|   '-- instructions.txt
|
'-- results/

```

The main Fortran source is stored inside the `src/` directory, while the EoS tables are stored under `eos/`. Numerical analysis and plotting are performed separately using Python scripts.

27.3 Program Input

The program obtains its run parameters from the file

`infile.`

A representative input is

```

1.e14   0.3e14   100
eos/EoS_dd2_npY_T0_beta_eq_eta_D-eta_V-Mg/\
EoS_dd2_npY_T0_beta_eq_01_04_Mg_800.dat

```

The first line contains

$$\rho_{c,\text{initial}}, \quad \Delta\rho_c, \quad N_{\text{models}}, \quad (27.2)$$

where

- $\rho_{c,\text{initial}}$ is the central total energy density of the first stellar model,
- $\Delta\rho_c$ is the central-density increment,
- N_{models} is the number of neutron-star configurations to calculate.

The corresponding section of the source code is

```
open(unit=1744,file='infile',
      status='unknown',form='formatted')

read(1744,*) rhoini, deltarho, nmodels

close(1744)
```

The second line specifies the equation-of-state table.

In the current version of the code, the filename is read using

```
read(1746,'(A)') eosfile
```

rather than list-directed input. This allows a relative path containing directory separators to be supplied, for example

```
eos/.../EoS_dd2_npY_T0_beta_eq_01_04_Mg_800.dat
```

without copying the EoS file into the repository root.

27.4 Main Program Variables

The principal dynamical variables are summarised in Table 3.

Variable	Physical quantity	Role
r	r	Radial coordinate
p	$P(r)$	Pressure
m	$m(r)$	Enclosed gravitational mass
mrest	$M_{\text{rest}}(r)$	Enclosed baryonic/rest mass
nue	$\nu(r)$	Metric potential
caph	$H(r)$	Tidal perturbation
beta	$H'(r)$	Derivative of tidal perturbation
rho0	ϵ_c	Central total energy density
pini	P_c	Central pressure
restini	$\rho_{\text{rest},c}$	Central rest-mass density
comp	C	Compactness M/R
tidalk2	k_2	Quadrupolar Love number
lambda	Λ	Dimensionless tidal deformability
y	y_R	Surface tidal quantity

Table 3: Principal variables used by the Fortran program.

27.5 Equation-of-State Arrays

The program allocates arrays of length 30000 for the EoS:

```
real, dimension(30000) ::  
    harray,  
    rhoarray,  
    parray,  
    earray,  
    ypparray,  
    narray,  
    e0array
```

For the currently used tabulated EoS format, each numerical row is read as

```
read(10,*,end=99) earray(i), parray(i),  
                  rhoarray(i), harray(i)
```

which corresponds to the ordering

$$\epsilon, \quad P, \quad n_B, \quad \mu_B. \quad (27.3)$$

The quantities are subsequently converted to the internal unit system used by the TOV integration.

27.6 Equation-of-State Initialisation

The EoS is loaded by the subroutine

```
eosinit2.
```

For the tabulated EoSs used in the present work, the program first converts energy density according to

```
earray(i)=earray(i)*1.783e12  
earray(i)=earray(i)/6.176e17
```

and similarly converts the pressure,

```
parray(i)=parray(i)*1.783e12  
parray(i)=parray(i)/6.176e17
```

The baryon number density is converted into a rest-mass density using the fiducial baryon mass

$$m_b = 933 \text{ MeV}, \quad (27.4)$$

implemented as

```
mb = 933.
```

```
rhoarray(i)=rhoarray(i)*mb
rhoarray(i)=rhoarray(i)*1.783e12
rhoarray(i)=rhoarray(i)/6.176e17
```

The program then stores the density, pressure and energy-density tables in logarithmic form,

```
rhoarray(i) = log10(rhoarray(i))
parray(i)    = log10(parray(i))
earray(i)    = log10(earray(i))
```

so that subsequent interpolation is performed in logarithmic space.

27.7 EoS Interpolation

Two principal interpolation directions are required.

27.7.1 Energy Density as a Function of Pressure

During the stellar integration, the TOV equations require

$$\epsilon = \epsilon(P). \quad (27.5)$$

The function

```
eps(pp)
```

calls

```
EosShen2D
```

to obtain the energy density corresponding to the current pressure.

The pressure is converted to logarithmic form,

```
dump = log10(pp)
```

and the code searches for the two neighbouring EoS points satisfying

$$P_{i-1} < P < P_i. \quad (27.6)$$

A linear interpolation is then performed in logarithmic space.

Schematically,

$$\log_{10} \epsilon(P) = \log_{10} \epsilon_{i-1} + \frac{\log_{10} P - \log_{10} P_{i-1}}{\log_{10} P_i - \log_{10} P_{i-1}} (\log_{10} \epsilon_i - \log_{10} \epsilon_{i-1}). \quad (27.7)$$

The result is transformed back using

$$\epsilon = 10^{\log_{10} \epsilon}. \quad (27.8)$$

27.7.2 Pressure as a Function of Energy Density

At the beginning of every stellar model, the central energy density is known and the corresponding central pressure must be obtained:

$$P_c = P(\epsilon_c). \quad (27.9)$$

This operation is performed by

```
pout(ee)
```

which calls the interpolation routine

```
poutshen.
```

Thus the two principal EoS calls are

$\text{eps}(P) \longrightarrow \epsilon(P),$ $\text{pout}(\epsilon) \longrightarrow P(\epsilon).$	(27.10)
--	---------

27.8 Central-Density Sequence

Before the sequence begins, the input central densities are converted using

```
rhoini    = rhoini/6.176e17
deltarho  = deltarho/6.176e17
```

and the first central density is initialised as

```
rho0 = rhoini
rho0step = deltarho
rho0 = rho0-rho0step
```

The main sequence is then generated by

```
do q=1,nmodels
    rho0=rho0+rho0step
    ...
enddo
```

Therefore the central density of model q is

$$\boxed{\rho_c^{(q)} = \rho_{c,0} + (q - 1)\Delta\rho_c.} \quad (27.11)$$

Each iteration of this outer loop represents an entirely new equilibrium neutron-star model.

27.9 Initial Conditions

The radial integration does not begin exactly at

$$r = 0, \quad (27.12)$$

because the differential equations contain singular-looking factors such as $1/r$ and $1/r^2$.

Instead, the program uses a small initial radius,

```
r0 = 10.*1e-5/1.746
h  = 10.*1e-5/1.746
```

where h is also the radial integration step.

The radial coordinate is initialised as

```
r = r0
```

and the enclosed masses are initially

```
m      = 0.
mrest  = 0.
```

The central pressure is obtained from the EoS:

```
p = pout(rho0)
```

```
pini    = p
restini  = rest(pini)
```

The initial gravitational mass is then estimated from the small central sphere,

```
m = 4.*pi*eps(p)*r0**3/3.
```

which corresponds to

$$\boxed{m(r_0) \simeq \frac{4\pi}{3}\epsilon_c r_0^3.} \quad (27.13)$$

27.10 Initial Tidal Perturbation

The $l = 2$ tidal perturbation is initialised using

```
a0    = 1.
caph  = a0*r*r
beta  = 2.*a0*r
```

Thus

$$H(r_0) = a_0 r_0^2, \quad (27.14)$$

and

$$H'(r_0) = 2a_0 r_0. \quad (27.15)$$

This is the regular central behaviour expected for an $l = 2$ perturbation.

27.11 Differential-Equation Routines

The program solves six coupled first-order differential equations. Their implementation is summarised in Table 4.

Routine	Returned quantity	Physical role
rhsp	dP/dr	TOV pressure equation
rhsm	dm/dr	Gravitational mass equation
rhsn	$d\nu/dr$	Metric-potential equation
rhsr	dM_{rest}/dr	Baryonic/rest-mass equation
rhsh	$dH/dr = \beta$	Tidal perturbation equation
rhsb	$d\beta/dr$	Second tidal perturbation equation

Table 4: Differential-equation routines in the Fortran program.

The mass equation is directly implemented as

```
rhsm = 4.*pi*rr**2*eps(pp)
```

corresponding to

$$\frac{dm}{dr} = 4\pi r^2 \epsilon. \quad (27.16)$$

The pressure equation is implemented by **rhsp**, including the relativistic TOV correction factors.

27.12 Tidal EoS Derivative

The tidal perturbation equation requires the derivative

$$\frac{d\epsilon}{dP}. \quad (27.17)$$

The code estimates this using a symmetric finite difference:

```
f = (eps(pp+0.01*pp)-eps(pp-0.01*pp))
    /(0.02*pp)
```

Therefore,

$$\boxed{\frac{d\epsilon}{dP} \simeq \frac{\epsilon(1.01P) - \epsilon(0.99P)}{0.02P}}. \quad (27.18)$$

The result enters the function

rhsb

which calculates the radial derivative of the tidal variable $\beta = H'$.

27.13 Fourth-Order Runge–Kutta Integration

All six dynamical variables are evolved simultaneously using a fourth-order Runge–Kutta scheme.

For the first RK stage, for example, the code calculates

```
k1 = rhsp(r,p,m)
l1 = rhsm(r,p,m)
m1 = rhsn(r,p,m)
n1 = rhsr(r,p,m)
o1 = rhsh(r,p,m,beta)
p1 = rhsb(r,p,m,beta,caph)
```

The correspondence is

$$k_i \longleftrightarrow \frac{dP}{dr}, \quad (27.19)$$

$$l_i \longleftrightarrow \frac{dm}{dr}, \quad (27.20)$$

$$m_i \longleftrightarrow \frac{d\nu}{dr}, \quad (27.21)$$

$$n_i \longleftrightarrow \frac{dM_{\text{rest}}}{dr}, \quad (27.22)$$

$$o_i \longleftrightarrow \frac{dH}{dr}, \quad (27.23)$$

$$p_i \longleftrightarrow \frac{d\beta}{dr}. \quad (27.24)$$

Four such stages are evaluated.

The full solution update is then

```

p = p + (h/6.)*(k1+2.*k2+2.*k3+k4)

m = m + (h/6.)*(l1+2.*l2+2.*l3+l4)

nue = nue + (h/6.)*(m1+2.*m2+2.*m3+m4)

mrest = mrest +
        (h/6.)*(n1+2.*n2+2.*n3+n4)

caph = caph +
        (h/6.)*(o1+2.*o2+2.*o3+o4)

beta = beta +
        (h/6.)*(p1+2.*p2+2.*p3+p4)

r = r+h

```

which is the standard RK4 update

$$\boxed{y_{j+1} = y_j + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4) .} \quad (27.25)$$

27.14 Surface and Termination Condition

The program integrates outward until the next RK4 pressure value would fall below a small threshold.

The implemented condition is

```

if (.not.(p+(h/6.)*
        (k1+2.*k2+2.*k3+k4)) > 6.e-17) exit

```

Thus the stellar surface is not determined by exactly

$$P = 0, \quad (27.26)$$

but by the numerical approximation

$$P \lesssim 6 \times 10^{-17} \quad (27.27)$$

in the internal pressure units.

The radius at this point is taken as the stellar radius R , while the integrated value of $\mathbf{m}$ is the gravitational mass M .

27.15 Metric Normalisation

After the integration reaches the surface, the metric potential is renormalised using

$$\text{nue0} = \log(\text{sqrt}(1.-2.*\text{G}*m/(r*c**2)))-\text{nue}$$

$$\text{nue} = \text{nue0}+\text{nue}$$

which ensures matching to the exterior Schwarzschild solution.

27.16 Calculation of the Tidal Surface Variable

At the stellar surface, the code calculates

$$y = r*\text{beta}/\text{caph} - 4.*\text{pi}*r*r*r*\text{eps}(p)/m$$

corresponding to

$$y_R = \frac{RH'(R)}{H(R)} - \frac{4\pi R^3 \epsilon(R)}{M}. \quad (27.28)$$

Since

$$\beta = H', \quad (27.29)$$

the ratio

$$\frac{R\beta}{H} \quad (27.30)$$

is the logarithmic derivative of the tidal perturbation at the surface.

27.17 Compactness

The program defines

$$\text{comp} = m/r$$

so that, in geometrized units,

$$C = \frac{M}{R}. \quad (27.31)$$

27.18 Love Number

The denominator of the quadrupolar Love-number expression is evaluated as

```
nenner =
  2.*comp*(6.-3.*y+3.*comp*(5.*y-8.))
+ 4.*comp**3*
  (13.-11.*y+comp*(3.*y-2.)
+2.*comp**2*(1.+y))
+ 3.*(1.-2.*comp)**2*
  (2.-y+2.*comp*(y-1.))
*log(1.-2.*comp)
```

and the Love number is then

```
tidalk2 =
  8.*comp**5/5.
  *(1.-2.*comp)**2
  *(2.+2.*comp*(y-1.))-y)
/nenner
```

which corresponds to the standard relativistic quadrupolar Love number

$$k_2 = k_2(C, y_R). \quad (27.32)$$

27.19 Dimensionless Tidal Deformability

The dimensionless tidal deformability is calculated directly as

```
lambda = 2./3.*tidalk2*(1./comp)**5
```

or

$$\Lambda = \frac{2}{3}k_2C^{-5}. \quad (27.33)$$

The program also calculates a tidal coupling constant assuming a symmetric binary with mass ratio $q = 1$:

```
kappa2 =
  2.*(1./2.**5 * tidalk2/comp**5
+ 1./2.**5 * tidalk2/comp**5)
```

27.20 Output File

The main output file is created using

```

open(unit=1745,
      file='logtov_seq_geom_tidal.dat',
      status='replace',
      form='formatted')

```

Because the file is opened with

```
status='replace',
```

each new program execution overwrites the previous `logtov_seq_geom_tidal.dat` file.

The output statement is

```

write(1745,*)
  rho0,
  1.476*r,
  m,
  mrest,
  exp(nue),
  nue0,
  (m/r),
  restini,
  pini,
  tidalk2,
  kappa2,
  lambda,
  y,
  eps(p)*6.176e17,
  caph,
  beta,
  ef,
  nf

```

Hence the current code writes 18 quantities for each stellar model.

Table 5: Columns of `logtov_seq_geom_tidal.dat`.

Column	Code quantity	Interpretation
1	<code>rho0</code>	Central total energy density
2	<code>1.476*r</code>	Stellar radius in km
3	<code>m</code>	Gravitational mass in $M_{\odot}$
4	<code>mrest</code>	Rest/baryonic mass
5	<code>exp(nue)</code>	Surface metric/lapse quantity
6	<code>nue0</code>	Metric normalisation constant
7	<code>m/r</code>	Compactness C
8	<code>restini</code>	Central rest-mass density
9	<code>pini</code>	Central pressure
10	<code>tidalk2</code>	Tidal Love number k_2
11	<code>kappa2</code>	Tidal coupling parameter
12	<code>lambda</code>	Dimensionless deformability Λ

Column	Code quantity	Interpretation
13	y	Surface tidal variable y_R
14	eps(p)*6.176e17	Surface energy density
15	caph	$H(R)$
16	beta	$H'(R)$
17	ef	Converted central energy quantity
18	nf	Interpolated baryon-density quantity

A particularly important version-specific point is that the current source writes

1.476*r

directly to column 2. Therefore the radius in the present output file is already in kilometres and must *not* be multiplied by 1.476 a second time during Python analysis.

27.21 Auxiliary Output Files

The program also generates

`profile.dat`

during the radial integration.

Every 100 integration steps, quantities including

$$r, \quad P, \quad \epsilon, \quad m, \quad M_{\text{rest}}, \quad \rho_{\text{rest}} \quad (27.34)$$

are written to this file.

The code additionally writes

`asshisto_tov.dat,`

which contains histogram-related quantities associated with the distribution of rest mass over density bins.

These files are diagnostic/generated outputs rather than primary input files.

27.22 Program Flow

The full program logic can be represented as

```
[ node distance=0.8cm, box/.style= draw, rounded corners, align=center, minimum
width=5.0cm, minimum height=0.75cm , >=stealth ]

[box] (input) Read infile
       $\rho_{c,0}$ ,  $\Delta\rho_c$ ,  $N$ , EoS path;

[box, below=of input] (eos) Read and convert EoS table
```

```

eosinit2;
[box, below=of eos] (model) Select next central density
 $\rho_c \leftarrow \rho_c + \Delta\rho_c$ ;
[box, below=of model] (pressure) Interpolate central pressure
 $P_c = P(\epsilon_c)$ ;
[box, below=of pressure] (init) Initialise
 $r, m, M_{\text{rest}}, \nu, H, \beta$ ;
[box, below=of init] (rk) RK4 integration of
 $P, m, \nu, M_{\text{rest}}, H, \beta$ ;
[box, below=of rk] (surface) Surface reached
 $P \lesssim P_{\text{threshold}}$ ;
[box, below=of surface] (tidal) Calculate
 $C, y_R, k_2, \kappa_2, \Lambda$ ;
[box, below=of tidal] (output) Write one row to
logtov_seq_geom_tidal.dat;
[box, below=of output] (repeat) Repeat for next central density;
[->] (input) - (eos); [->] (eos) - (model); [->] (model) - (pressure); [->] (pressure) - (init);
[->] (init) - (rk); [->] (rk) - (surface); [->] (surface) - (tidal); [->] (tidal) - (output); [->]
(output) - (repeat);

```

27.23 Compilation

From the repository root, the current source can be compiled with

```

gfortran -o logtov_seq_geom_tidal.out \
src/logtov_seq_geom_tidal.f90 \
-fno-automatic

```

The resulting executable is

```
logtov_seq_geom_tidal.out.
```

Since this executable is platform dependent, it should generally not be stored in version control. Instead, the source should be compiled on the machine on which the calculation will be run.

27.24 Running the Code

After compilation, the executable is run from the repository root:

```
./logtov_seq_geom_tidal.out
```

The working directory must contain the `infile`, and the relative EoS path written inside `infile` must be valid from that working directory.

After a successful calculation, the main results are contained in

```
logtov_seq_geom_tidal.dat.
```

27.25 Python Analysis

The output can be loaded using NumPy:

```
import numpy as np

data = np.loadtxt("logtov_seq_geom_tidal.dat")

rho_geom = data[:, 0]
R         = data[:, 1]
M         = data[:, 2]
compact   = data[:, 6]
k2        = data[:, 9]
Lambda    = data[:, 11]
```

The physical central density in g cm^{-3} is obtained from

```
rho_c = rho_geom * 6.176e17
```

or

$$\rho_c[\text{g cm}^{-3}] = \rho_{\text{output}} \times 6.176 \times 10^{17}. \quad (27.35)$$

For the current version of the source,

```
R = data[:,1]
```

is already in kilometres.

27.26 Important Implementation Notes

Several implementation details should be kept in mind when interpreting the numerical results.

27.26.1 EoS Header Handling

The EoS initialisation routine contains

```
read(10,'(A)') adum
```

before reading numerical EoS points.

The code therefore assumes that the first line of the EoS file is a header and discards it.

If an EoS table begins directly with numerical data, the first numerical row will instead be skipped.

This should be verified for every EoS format used.

27.26.2 Low-Mass Tidal Models

The low-central-density portion of the stellar sequence can produce unusual or numerically unstable tidal quantities.

In the tested sequence, some low-mass configurations produced negative values of k_2 and Λ .

Such configurations should not automatically be interpreted as physical tidal responses.

Particular attention should therefore be paid to

$$k_2 > 0, \quad \Lambda > 0, \quad (27.36)$$

and to the behaviour of the mass-radius sequence before performing astrophysical comparisons.

27.26.3 Output Overwriting

The output files are opened using

```
status='replace'.
```

A new run therefore overwrites the previous results.

When comparing multiple EoSs, each output should be copied or renamed after the run, for example

```
cp logtov_seq_geom_tidal.dat \
results/EoS_01_04_Mg800.dat
```

before running the next EoS.

27.26.4 Radial Step Size

The integration step is currently specified directly in the source by

```
h = 10.*1e-5/1.746
```

Changing this quantity alters both computational cost and numerical accuracy.

Convergence should therefore be checked by repeating selected stellar models with smaller values of h .

This is particularly important if numerical derivatives of the mass-radius relation are required.

27.27 Suggested Future Improvements

For continued development, several improvements would make the code more robust and easier to maintain:

1. Move the radial step size and surface-pressure threshold from the source code into `infile`.

2. Store all runtime configuration in a dedicated input structure rather than reopening `infile` in several routines.
3. Add explicit checks that the EoS file exists and that its pressure and energy-density columns are monotonic.
4. Remove the unconditional first-line discard, or make EoS header handling configurable.
5. Use the actual EoS table length during interpolation rather than the full allocated array size.
6. Add bounds checking for every EoS lookup.
7. Write a header line to `logtov_seq_geom_tidal.dat` describing each output column.
8. Store numerical constants such as π , unit-conversion factors and the baryon mass as named parameters.
9. Separate the TOV equations, EoS routines and tidal equations into individual Fortran modules.
10. Add automated convergence tests for the radial step size.
11. Add automated extraction of $M_{\max}$, $R_{1.4}$, $\Lambda_{1.4}$, and other comparison quantities.

27.28 Summary of Code Execution

The operation of the program can be summarised as

Read EoS $\rightarrow$ convert and interpolate $\rightarrow$ choose ϵ_c $\rightarrow P_c = P(\epsilon_c)$ $\rightarrow$ initialise central solution $\rightarrow$ RK4 integration $\rightarrow P(R) \simeq 0$ $\rightarrow M, R, C, y_R$ $\rightarrow k_2, \Lambda$ $\rightarrow$ write output $\rightarrow$ repeat for next ϵ_c .	(27.37)
---	---------

28 References

References

- [1] J. R. Oppenheimer and G. M. Volkoff, *On Massive Neutron Cores*, Physical Review **55**, 374 (1939).
- [2] R. C. Tolman, *Static Solutions of Einstein's Field Equations for Spheres of Fluid*, Physical Review **55**, 364 (1939).

- [3] T. Hinderer, *Tidal Love Numbers of Neutron Stars*, The Astrophysical Journal **677**, 1216 (2008).
- [4] T. Hinderer, B. D. Lackey, R. N. Lang, and J. S. Read, *Tidal Deformability of Neutron Stars with Realistic Equations of State and Their Gravitational Wave Signatures in Binary Inspiral*, Physical Review D **81**, 123016 (2010).